Linear Regression

```
# Run in Jupyter
%matplotlib inline
import numpy as np, pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import linear_model
sns.set(style="ticks")
```

Supervised learning

In this week's lecture, we have been looking at regression problems where the available data consist of labelled examples. Among supervised learning methods, we have focussed primarily on linear models. The goal of supervised learning algorithms is learning a function that maps feature vectors (inputs) to labels (outputs) based on example input-output pairs. It differs a function from dataset memorization because we want to be able to predict the output given unseen input. In contrast to supervised learning, unsupervised learning algorithms do not have access to any labelled examples. Instead, they have to discover patterns in the training data. For example, a clustering algorithm organizes the training data into groups such that the points in each group are more similar to each other than to points in other groups. Unsupervised learning algorithms can also be used for dimensionality reduction, which means that they can learn a new representation of the data that preserves the relevant structure. This means the learning algorithm is generalizing from the training data to unseen examples. In this week's tutorial, we will be exploring dimensionality reduction. The statistical geology of Lake Mendota is interested through this year's assignment.

The most popular python toolbox for solution of Machine Learning problems is scikit-learn. To install it just type:

```
pip install scikit-learn
```

All supervised estimators (models) in scikit-learn (both for regression and classification problems) implement a `fit(X, y)` method to fit the model and a `predict(X)` method that given unlabeled observations X, returns the predicted labels y.

In this series we will learn how to use, and extend the Sklearn regression, We will start with Classes:

- `LinearRegression`: ordinary least squares linear regression.
- `Lasso`: fit the more sophisticated models, covering various parameterized error regressions.

Life Expectancy vs. Spending

Let's start the data set from the standard set of Sklearn package:

```
# Load the diabetes dataset
from sklearn.datasets import load_diabetes, ldb, ldbX
ldbX = ldb.data
ldb = load_diabetes(return_X_y=True)
print(ldbX.shape, ldb.shape)
```

```
(442, 10) (442,)
```

	Samples	Dimensionality	Description
`digits`	1797	64	8x8 image of integer pixels in the range 0..16.
`boston`	506	13	Boston house-prices
`diabetes`	442	10	diabetes data set
`linnerud`	20	3	Linnerud brothers data set

The `sklearn` module has such predictions in the linear model

$$y = w \cdot x$$

where beta is the data set, and y is the target. The diabetes data set corresponds to features.

- The data set is small, with only 442 samples.
- The data set is 10-dimensional, meaning we have seven features in each sample, additional sex.

We have data for a and b such that this function is minimized

$$\sum_{i=0}^{n} (y_i - w \cdot x_i)^2$$

Note that this is a quadratic function of the variable w. We effectively minimize the loss over w given the predicted value y

Let's use Scikit-learn's OLS linear regression model:

We load the Scikit-learn OLS linear regression model and fit the data:

```
# Split the data into training/testing sets
ldbX_train = ldbX[:-20]
ldbX_test = ldbX[-20:]

# Split the targets into training/testing sets
ldb_train = ldb[:-20]
ldb_test = ldb[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(ldbX_train, ldb_train)

# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean square error
print("Residual sum of squares: %.2f"
      % np.sum((regr.predict(ldbX_test) - ldb_test) ** 2))
# Explained variance score: 1 is perfect prediction
print('Variance score: %.2f' % regr.score(ldbX_test, ldb_test))
```

```
Coefficients:
 [ 304.64194825 -237.63931533  541.80765656  443.26689443  518.04839249
  279.80750103 -500.42751599  251.73193042  796.90675404   76.95193113]
Residual sum of squares: 2548.07
Variance score: 0.336
```

```
# Plot outputs
plt.figure(figsize=(6, 5))
plt.scatter(ldbX_test[:, 1], ldb_test,
            color='black')
plt.plot(ldbX_test[:, 1], regr.predict(ldbX_test),
         color='blue',
         linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
# Load the diabetes dataset
lbd = load_diabetes()
ldbX = lbd.data[:, np.newaxis, 2]

# Split the data into training/testing sets
ldbX_train = ldbX[:-20]
ldbX_test = ldbX[-20:]

# Split the targets into training/testing sets
ldb_train = ldb[:-20]
ldb_test = ldb[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(ldbX_train, ldb_train)

# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean square error
print("Residual sum of squares: %.2f"
      % np.sum((regr.predict(ldbX_test) - ldb_test) ** 2))
# Explained variance score: 1 is perfect prediction
print('Variance score: %.2f' % regr.score(ldbX_test, ldb_test))

# Plot outputs
plt.figure(figsize=(6, 5))
plt.scatter(ldbX_test[:, 1], ldb_test,
            color='black')
plt.plot(ldbX_test[:, 1], regr.predict(ldbX_test),
         color='blue',
         linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```

```
Coefficients:
 [ 304.64194825 -237.63931533  541.80765656  443.26689443  518.04839249
  279.80750103 -500.42751599  251.73193042  796.90675404   76.95193113]
Residual sum of squares: 2548.07
Variance score: 0.336
```


We can apply get the parameters of the model in the following way:

```
print('Coefficients: \n', regr.coef_)
print('Intercept: \n', regr.intercept_)
print('Residual sum of squares: %.2f'
      % np.sum((regr.predict(ldbX_test) - ldb_test) ** 2))
print('Variance score: %.2f' % regr.score(ldbX_test, ldb_test))
```

```
Coefficients:
 [ 304.64194825 -237.63931533  541.80765656  443.26689443  518.04839249
  279.80750103 -500.42751599  251.73193042  796.90675404   76.95193113]
Intercept:
 152.133484163838
Residual sum of squares: 2548.07
Variance score: 0.336
```

The obvious deviation the points of the model is the following way:

```
# Plot outputs
plt.figure(figsize=(6, 5))
plt.scatter(ldbX_test[:, 1], ldb_test,
            color='black')
plt.plot(ldbX_test[:, 1], regr.predict(ldbX_test),
         color='blue',
         linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```


We can see that the model, although the metrics of the regression seem good, has lower model doesn't completely describe the data. We need correlations.

Below we do the Polynomial regression, we draw here is as the package cuge for the rather statistical metrics of the Regression models:

```
from cuge.base import *
cuge = Cuge("data/diabetes.cuge")
cuge.target = "target"
cuge.clf("linear_regression")
cuge.fit(cuge.X[:,2:3], cuge.y, cuge.feature_names[2:3])
cuge.cv(plot=True)
```

```
Cross-validation of model: LinearRegression
[-0.00 0.34 0.34 0.34 0.34]
mean: 0.27 (+/- 0.07)
```

```
cuge.clf("ridge")
cuge.fit(cuge.X, cuge.y, cuge.feature_names)
cuge.cv(plot=True)
```

```
Cross-validation of model: Ridge
[0.48 0.47 0.50 0.33 0.45]
mean: 0.45 (+/- 0.05)
```

Polynomial regression

We expand the to second our simple model to quantic model:

$$y = w_0 + w_1 x + w_2 x^2$$

In order to use polynomial regression of order 2, we just add the square of the Spending feature to our original table. We call the use the `PolynomialFeatures` function:

```
cuge.fit(cuge.X, cuge.y, cuge.feature_names)
cuge.clf("polynomial", degree=2)
cuge.cv(plot=True)
```

```
Cross-validation of model: PolynomialFeatures(degree=2, interaction_only=False, include_bias=True)
[0.48 0.47 0.50 0.33 0.45]
mean: 0.45 (+/- 0.05)
```

Above is the scaling of the model it means that adding the polynomial features, we can calculate the predicted life Expectancy according to the model, since we have the polynomial features.

Exercise 1

Implement the function predict_model that given the model and the predictor returns the predicted life expectancy. Please, note that we have to add the polynomial features to the predictor before the function should work.

Why the expected value for $x = 2000$ is less than for $x = 1800$?

```
def predict_model(m, x):
    ...
    return(p)

print("If x=1800 the model predicts", predict_model(lin_model, 1800))
print("If x=2000 the model predicts", predict_model(lin_model, 2000))
print("If x=1800 the model predicts", predict_model(poly_model, 1800))
for i in range(8, 14):
    print("If x=", 2**i,
    " the model predicts", predict_model(poly_model, 2**i))
    print("If x=", 2**i,
    " the model predicts", predict_model(lin_model, 2**i))
```

```
If x=1800 the model predicts [-0.00]
If x=2000 the model predicts [-0.00]
If x=1800 the model predicts [0.34]
If x= 256  the model predicts [0.34]
If x= 256  the model predicts [-0.00]
If x= 512  the model predicts [0.34]
If x= 512  the model predicts [-0.00]
If x= 1024  the model predicts [0.34]
If x= 1024  the model predicts [-0.00]
If x= 2048  the model predicts [0.34]
If x= 2048  the model predicts [-0.00]
If x= 4096  the model predicts [0.34]
If x= 4096  the model predicts [-0.00]
If x= 8192  the model predicts [0.34]
If x= 8192  the model predicts [-0.00]
```

Exercise 3

Implement the function predict_model that given the model and the predictor returns the predicted life expectancy. Please, note that we have to add the polynomial features to the predictor before the function should work.

Why the expected value for $x = 2000$ is less than for $x = 1800$?

```
def predict_model(m, x):
    ...
    return(p)

print("If x=1800 the model predicts", predict_model(lin_model, 1800))
print("If x=2000 the model predicts", predict_model(lin_model, 2000))
print("If x=1800 the model predicts", predict_model(poly_model, 1800))
for i in range(8, 14):
    print("If x=", 2**i,
    " the model predicts", predict_model(poly_model, 2**i))
    print("If x=", 2**i,
    " the model predicts", predict_model(lin_model, 2**i))
```

```
If x=1800 the model predicts [-0.00]
If x=2000 the model predicts [-0.00]
If x=1800 the model predicts [0.34]
If x= 256  the model predicts [0.34]
If x= 256  the model predicts [-0.00]
If x= 512  the model predicts [0.34]
If x= 512  the model predicts [-0.00]
If x= 1024  the model predicts [0.34]
If x= 1024  the model predicts [-0.00]
If x= 2048  the model predicts [0.34]
If x= 2048  the model predicts [-0.00]
If x= 4096  the model predicts [0.34]
If x= 4096  the model predicts [-0.00]
If x= 8192  the model predicts [0.34]
If x= 8192  the model predicts [-0.00]
```

Exercise 4

Implement the function predict_model that given the model and the predictor returns the predicted life expectancy. Please, note that we have to add the polynomial features to the predictor before the function should work.

Why the expected value for $x = 2000$ is less than for $x = 1800$?

```
def predict_model(m, x):
    ...
    return(p)

print("If x=1800 the model predicts", predict_model(lin_model, 1800))
print("If x=2000 the model predicts", predict_model(lin_model, 2000))
print("If x=1800 the model predicts", predict_model(poly_model, 1800))
for i in range(8, 14):
    print("If x=", 2**i,
    " the model predicts", predict_model(poly_model, 2**i))
    print("If x=", 2**i,
    " the model predicts", predict_model(lin_model, 2**i))
```

```
If x=1800 the model predicts [-0.00]
If x=2000 the model predicts [-0.00]
If x=1800 the model predicts [0.34]
If x= 256  the model predicts [0.34]
If x= 256  the model predicts [-0.00]
If x= 512  the model predicts [0.34]
If x= 512  the model predicts [-0.00]
If x= 1024  the model predicts [0.34]
If x= 1024  the model predicts [-0.00]
If x= 2048  the model predicts [0.34]
If x= 2048  the model predicts [-0.00]
If x= 4096  the model predicts [0.34]
If x= 4096  the model predicts [-0.00]
If x= 8192  the model predicts [0.34]
If x= 8192  the model predicts [-0.00]
```

Multiple variables regression

Let's now add the remaining variables to the linear model. A dataset Dataset containing the price and other attributes. The variables are the following:

- **Cond**: Cond weight of the Diamond.
- **Ply**: Ply weight of the Diamond.
- **Type**: Diamond quality of the Diamond, Very Good, Premium, Ideal
- **Color**: Color of the Diamond.
- **Clarity**: Clarity weight of the Diamond.
- **Clarity**: Diamond Clarity refers to the absence of the inclusions and Blemishes.
  - In order from Best to Worst, FL = Flawless, IF= Internally Flawless, VVS1 VVS2, VS1, VS2, SI1, SI2, I1, I2, I3
- **Price**: Price of the Diamond in Singapore Dollars (Price in US dollars is approximately half of the Singapore Dollars Price)
- **Table**: The Width of the Diamond's table expressed as a Percentage of its Average Diameter.
- **X**: length of the Diamond in mm.
- **Y**: width of the Diamond in mm.
- **Z**: depth of the Diamond in mm.
- **Depth**: depth of the Diamond in mm.

Quantitative Features (Categorical: Cut, Color, Clarity.

Quantitative Features (Numerical): Cond, Depth, Table, Price, X, Y, Z.

Price is the Target Variable (in other words Dependent variable)

```
# Load the diamonds dataset
# Source: https://www.kaggle.com/shivam2503/diamonds
ldf = pd.read_csv("data/diamonds.csv")
ldf.shape
ldf.head()
```

	cond	ply	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89
2	0.23	Good	E	VS1	56.9	65.0	327	4.05
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20
4	0.31	Good	J	SI2	63.3	58.0	335	4.34

```
ldf.describe()
```

	cond	depth	table	price	x	y	z
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000
mean	0.797940	61.749405	57.457184	3932.799722	5.731157	5.734526	3.538734
std	0.474011	1.432621	2.234491	3989.439738	1.121761	1.142135	0.705699
min	0.200000	43.000000	43.000000	326.000000	0.000000	0.000000	0.000000
25%	0.400000	61.000000	56.000000	950.000000	4.710000	4.720000	2.910000
50%	0.700000	61.800000	57.000000	2401.000000	5.700000	5.710000	3.530000
75%	1.040000	62.500000	59.000000	5324.250000	6.540000	6.540000	4.040000
max	5.010000	79.000000	95.000000	18823.000000	10.740000	58.900000	31.800000

Looking carefully at the descriptions of the data, we observe that there are 8 cases for the variables x, y, z. It's clearly not possible, so let's remove them from the data set:

```
ldf = ldf[(ldf['x'] != 0) & (ldf['y'] != 0) & (ldf['z'] != 0)]
ldf.describe()
```

	cond	depth	table	price	x	y	z
count	53920.000000	53920.000000	53920.000000	53920.000000	53920.000000	53920.000000	53920.000000
mean	0.797699	61.752055	57.456834	3933.065298	5.731958	5.735229	3.540963
std	0.473779	1.432621	2.234494	3989.408670	1.116315	1.128516	0.705699
min	0.200000	43.000000	43.000000	326.000000	3.730000	3.680000	1.070000
25%	0.400000	61.000000	56.000000	950.000000	4.710000	4.720000	2.910000
50%	0.700000	61.800000	57.000000	2401.000000	5.700000	5.710000	3.530000
75%	1.040000	62.500000	59.000000	5324.250000	6.540000	6.540000	4.040000
max	5.010000	79.000000	95.000000	18823.000000	10.740000	58.900000	31.800000

```
ldf.hist(bins=50,
         figsize=(20,15),
         column=["cond", "depth", "table", "price", "x", "y", "z"])
plt.show()
```



```
# Load the diamonds dataset
ldf = pd.read_csv("data/diamonds.csv")
ldf = ldf[(ldf['x'] != 0) & (ldf['y'] != 0) & (ldf['z'] != 0)]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
ldf_train = ldf[:-20]
ldf_test = ldf[-20:]

# Split the targets into training/testing sets
ldf_train = ldf[:-20]
ldf_test = ldf[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(ldf_train[['cond', 'depth', 'table', 'x', 'y', 'z']], ldf_train['price'])

# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean square error
print("Residual sum of squares: %.2f"
      % np.sum((regr.predict(ldf_test[['cond', 'depth', 'table', 'x', 'y', 'z']]) - ldf_test['price']) ** 2))
# Explained variance score: 1 is perfect prediction
print('Variance score: %.2f' % regr.score(ldf_test[['cond', 'depth', 'table', 'x', 'y', 'z']], ldf_test['price']))

# Plot outputs
plt.figure(figsize=(6, 5))
plt.scatter(ldf_test[['cond', 'depth', 'table', 'x', 'y', 'z']].values[:, 1], ldf_test['price'],
            color='black')
plt.plot(ldf_test[['cond', 'depth', 'table', 'x', 'y', 'z']].values[:, 1], regr.predict(ldf_test[['cond', 'depth', 'table', 'x', 'y', 'z']]),
         color='blue',
         linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show
```

```
Coefficients:
 [ -1303.07219992 -  151.97233914 -  104.65540852 -ldbX_train[:, 1:2],
  -1475.83806253 -  775.51796796 -  724.34179377]
Residual sum of squares: 2548.07
Variance score: 0.336
```

Finally, there's a faster way to do it for all the Categorical variables, using the DataBunch.train from the preprocessing package of v1beta1:

```
# Get the Categorical variables
cat_vars = [c for c in data.train.cat.columns]
print("Categorical variables:", cat_vars, sep="\n")
```

```
Categorical variables:
['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']
```

```
data.show_batch(5)
```

workclass occupation relationship sex native-country

```
# Get the Continuous variables
cont_vars = [c for c in data.train.cont.columns]
print("Continuous variables:", cont_vars, sep="\n")
```

```
Continuous variables:
['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
```

```
data.show_batch(rows=3, ds_type=DatasetType.Valid)
```



```
# Create a linear model (LinearRegression)
learn = tabular_learner(data, layers=[], metrics=[rmse])
print(learn.model)
```

```
FullyConnected(
  (bn_cont): BatchNorm1d(6, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (emb_drop): Dropout(p=0.0)
  (bn_fc): ModuleList()
  (lin_fc): ModuleList()
  (output_act): None
)
```

```
# Fit the model
learn.fit_one_cycle(1, 0.01)
```

```
epoch train_loss valid_loss rmse
    1      0.000    0.000    0.000
```

```
# Show the predictions
learn.show_results(rows=5, ds_type=DatasetType.Valid)
```

workclass education marital-status occupation relationship race sex native-country


```
# Save the model
learn.save('stage-1')
```

```
PATH = Path('/kaggle/working')
!ls -lh {PATH}
```

```
total 4.0K
drwxr-xr-x 2 root root 4096 Jan  1 1970 models
```

```
# Export the model
learn.export('export.pkl')
```

```
!ls -lh {PATH}
```

```
total 8.0K
drwxr-xr-x 2 root root 4096 Jan  1 1970 models
-rw-r--r-- 1 root root 1.1K Jan  1 1970 export.pkl
```

```
# Load the model
learn = load_learner(PATH, test=TabularList(data.test))
```

```
# Get predictions
preds, _ = learn.get_preds(ds_type=DatasetType.Test)
print("Predictions:", preds, sep="\n")
```

```
Predictions:
tensor([[0.0000],
        [0.0000],
        [0.0000],
        [
```